

Luo

**OpenDRIVE-Based SUMO-CARLA Co-Simulation for Mobility and Energy Assessment in Urban
Transportation Networks**

Xiangyong Luo*

Researcher

Independent Researcher

United States

Email: luoxiangyong01@gmail.com

Total Number of Pages: 9

Submitted May 31, 2026

*Corresponding Author

ABSTRACT

Objectives: Urban traffic operations and digital-twin studies need consistent road networks across microscopic traffic simulation and high-fidelity driving simulation. This study develops an OpenDRIVE-centered workflow using PyOpenDRIVE, Simulation of Urban MObility (SUMO), and the CARLA driving simulator to support reproducible mobility and vehicle energy assessment from a shared lane-level network.

Methods: PyOpenDRIVE is used to inspect and edit an OpenDRIVE road network, write a reproducible project directory, and generate SUMO-CARLA co-simulation inputs from a shared .xodr file. The workflow creates SUMO network, route, vehicle-type, configuration, and run files; CARLA scripts for loading the same OpenDRIVE network and collecting vehicle telemetry; and optional SUMO-CARLA synchronization scripts. The case study uses the repository's Tempe, Arizona, OpenDRIVE and SUMO files and evaluates a 10-minute smoke-run window. Outputs are analyzed with streaming XML and comma-separated-value parsers that write reproducible tables, metadata, and figures.

Findings: The implemented workflow prepared a Tempe OpenDRIVE network containing 5,885 roads, 755 junctions, 15,496 lanes, 4,149 signals, and 243 controllers. A SUMO-only validation run completed two selected vehicles and produced reproducible mobility summaries, command logs, and high-resolution figures. CARLA and co-simulation results were not statistically tested in this smoke run and are explicitly reported as unavailable when the external CARLA runtime is not executed.

Novelty: The study contributes a reproducible OpenDRIVE-based workflow that links 3D real-world network editing, lane-level preparation, SUMO mobility-energy simulation setup, CARLA telemetry setup, and transportation-ready result exports.

Practical Applications: Transportation agencies and researchers can use the workflow to prepare consistent digital-twin simulation cases, document simulator inputs, and screen traffic operations, electrification, and control scenarios with transparent reproducibility records.

1 INTRODUCTION

2 Urban transportation networks are increasingly evaluated through simulation before field deployment of signal
3 timing plans, connected-vehicle strategies, electric-fleet policies, or digital-twin applications. These studies often need
4 two levels of representation. A network-scale traffic simulator is needed for vehicle movements, delay, emissions, and
5 energy use over many trips. A high-fidelity driving simulator is useful for three-dimensional visualization, vehicle
6 telemetry, and automated-driving experiments. The practical difficulty is that these tools can require different network
7 formats, different coordinate assumptions, and different execution scripts. A study can therefore lose reproducibility
8 when the same road network is edited separately for each simulator.

9 OpenDRIVE addresses part of this problem by providing a standardized static road network description
10 for simulation, including roads, reference lines, lanes, road markings, objects, and signals (ASAM e.V. 2026).
11 SUMO provides a widely used microscopic traffic simulation platform for mobility, emission, and energy analysis
12 (Álvarez López et al. 2018). CARLA provides an open urban driving simulator for high-fidelity autonomous-driving
13 and vehicle telemetry experiments (Dosovitskiy et al. 2017). However, an applied transportation study still needs a
14 repeatable workflow that prepares one OpenDRIVE network, generates the SUMO and CARLA project files, records
15 the exact commands, and exports analysis tables that can be reviewed.

16 This paper presents PyOpenDRIVE as an OpenDRIVE-centered workflow layer for joint SUMO and CARLA
17 studies. PyOpenDRIVE is a Python package adapted from the libOpenDRIVE project and extended with OpenDRIVE
18 reading, inspection, editing, SUMO conversion, browser visualization, and a simulation project builder (libOpenDRIVE
19 Contributors 2023). The contribution of this paper is not a new traffic flow model or a new vehicle energy model.
20 Instead, the contribution is a reproducible workflow that makes simulator preparation and result processing auditable
21 for transportation studies.

22 The study has three objectives. First, it defines an OpenDRIVE-centered project layout that keeps network
23 files, simulator inputs, commands, outputs, and analysis artifacts together. Second, it implements SUMO, CARLA,
24 and optional SUMO-CARLA co-simulation builders that can run in dry-run mode when external simulators are not
25 installed. Third, it demonstrates the workflow on a Tempe urban network example and reports the generated artifacts
26 and SUMO-only mobility outputs.

27 BACKGROUND

28 OpenDRIVE for Simulation Network Exchange

29 OpenDRIVE stores a road network as an XML-based static description with the file extensions `.xodr` and
30 `.xodr.z`. The current ASAM standard describes its application to road-network description, driving simulation, and
31 traffic simulation (ASAM e.V. 2026). The format is useful for this study because it can represent lane-level network
32 geometry and traffic-control features in one exchange file. That representation does not remove all interoperability
33 problems. SUMO and CARLA may still interpret geometry, junctions, lane links, and signals differently. A reproducible
34 workflow must therefore preserve the source file, record any prepared copy, and report which simulator-specific files
35 were generated from it.

36 SUMO and CARLA Roles

37 SUMO is used in this workflow as the primary source of traffic mobility, emission, and electric-vehicle energy
38 outputs. Its network conversion, route generation, and vehicle-type files make it suitable for scenario studies that vary
39 demand, fleet mix, speed assumptions, and traffic operations. The workflow creates SUMO configuration files that can
40 request trip information, emissions, battery outputs, summary statistics, and floating-car data when those outputs are
41 needed (Eclipse SUMO Project 2026).

42 CARLA is used as the high-fidelity simulation and telemetry layer. The workflow generates scripts that load
43 the same prepared OpenDRIVE network into CARLA standalone mode, apply a fixed simulation time step, and collect
44 vehicle positions, orientations, speeds, accelerations, and jerk values. CARLA is not treated as the default energy
45 model in this workflow unless a separate validated energy model is added. The optional co-simulation scripts follow
46 the CARLA SUMO-synchronization pattern and assume that the CARLA server and helper scripts are installed outside
47 the Python package (CARLA Simulator Project 2026).

48 Recent OpenDRIVE and SUMO-CARLA Co-Simulation Studies

49 Recent literature shows why OpenDRIVE-centered preparation remains important for transportation simu-
50 lation. Simulator surveys emphasize CARLA’s role as an open-source, high-fidelity 3D environment and identify
51 co-simulation with traffic simulators as a key capability for autonomous-driving research (Li et al. 2024; Silva et al.

2024). A recent connected-and-automated vehicle co-simulation survey similarly frames coupled traffic, vehicle, perception, and communication models as a response to the limits of single-platform simulation (Ashayer et al. 2025). Application studies have used CARLA-SUMO co-simulation for cooperative platooning and merging (Han et al. 2022), the OpenCDA framework and research ecosystem (Xu et al. 2021, 2023), simulation-to-real CDA integration (Zheng et al. 2023), infrastructure-assisted trajectory prediction and risk assessment (Cao et al. 2025), roadside LiDAR placement (Chen et al. 2024), camera-based traffic-signal reinforcement learning (Azfar et al. 2025), and performance scaling with mesoscopic traffic simulation (Varga et al. 2023). Verification work such as TSC2CARLA also demonstrates growing interest in toolchains that translate abstract driving scenarios into CARLA executions (Borchers et al. 2025). Across these studies, OpenDRIVE and related high-definition map representations provide the static road-network foundation, while SUMO and CARLA divide network-scale mobility from high-fidelity vehicle and sensor simulation. The remaining practical gap is a reproducible network-preparation layer that can inspect, edit, export, and document the shared OpenDRIVE source before simulator-specific files are produced.

METHODOLOGY

3.1 PyOpenDRIVE: The 3D Real-World OpenDRIVE Network Viewer and Editor

The central method in this study is the development of PyOpenDRIVE as a 3D real-world OpenDRIVE network viewer and editor. Existing online OpenDRIVE viewers are useful for loading an `.xodr` file, visualizing the road model in a browser, and in some cases exporting a static 3D mesh. PyOpenDRIVE extends that viewer concept into an editable, georeferenced, transportation network preparation environment. In this sense, PyOpenDRIVE is positioned as a state-of-the-art OpenDRIVE web viewer/editor for this workflow: it combines OpenDRIVE parsing, lane-level visualization, real-world basemap alignment, interactive network editing, and `.xodr`-based SUMO-CARLA co-simulation environment generation in one reproducible toolchain.

The viewer/editor is implemented with a local Python HTTP server and a MapLibre GL front end. The server reads an OpenDRIVE file through PyOpenDRIVE, exports roads, lane polygons, traffic signals, posts, mast arms, and other supported objects as GeoJSON, and converts OpenDRIVE local coordinates to longitude and latitude with `convertXY2LonLat`. When the analyst saves edits, the server applies the reverse conversion with `convertLonLat2XY` and writes supported geometry and attribute changes back to OpenDRIVE XML. This design keeps the browser responsive while keeping the OpenDRIVE parsing and XML update logic in Python, where it can be tested and reused by the simulation builder.

The editor is designed for real-world network inspection rather than a detached schematic view. When the input network contains real-world longitude and latitude information, the map can be displayed on road, satellite, or terrain basemaps; otherwise, it falls back to a grid-mesh view for local-coordinate networks. Users can click and drag lanes, signal heads, signal posts, mast arms, and environment objects, inspect selected attributes in the `OpenDriveViewer` panel, edit lane identifiers and predecessor/successor relationships, add or delete lanes and signal-support objects, undo the most recent movement, and save an `edited_network.xodr` file through the `/api/save-xodr` endpoint. The save path is intentionally conservative: it updates supported road, lane, signal, and object attributes while preserving OpenDRIVE content that is not handled by the editor.

This viewer/editor makes PyOpenDRIVE more than a file converter. Starting from one source `.xodr` file, PyOpenDRIVE can build the working directory for a SUMO-CARLA co-simulation case: a prepared OpenDRIVE network, SUMO network and route inputs, vehicle-type and simulation-configuration files, CARLA scripts for OpenDRIVE world loading and telemetry collection, and synchronization scripts for CARLA’s SUMO bridge. The viewer/editor provides the reviewable preparation layer where analysts can verify lane-level geometry, traffic-control assets, object placement, and coordinate alignment before those simulator files are created. For a TRB-style workflow, the same design supports reproducibility because the source `.xodr`, edited `.xodr`, generated simulator inputs, run commands, and analysis artifacts can be stored in the same project directory.

Workflow Design

The workflow starts with one OpenDRIVE `.xodr` file and creates a project directory with separate folders for the prepared network, SUMO files, CARLA files, co-simulation scripts, and analysis outputs. This structure turns the `.xodr` network into an executable SUMO-CARLA co-simulation environment when the external SUMO and CARLA dependencies are available, and into a fully auditable dry-run environment when they are not. Table 1 summarizes the main components. Each build writes a `project_config.json` file containing the source network path, scenario configuration, build flags, and dry-run status. This file is the main reproducibility record for the generated case.

TABLE 1 Workflow Components and Reproducibility Files

Component	Main function	Reproducibility files
OpenDRIVE preparation	Copy source .xodr, create prepared network, and summarize roads, lanes, signals, and controllers	network/input.xodr, network/prepared.xodr, network/network_summary.json
SUMO builder	Create or copy SUMO network, route, vehicle-type, configuration, and run files	sumo/scenario.sumocfg, sumo/vtypes.add.xml, sumo/build_sumo_plan.json
CARLA builder	Write scripts to load the prepared OpenDRIVE network and collect vehicle telemetry	carla/load_opendrive_world.py, carla/collect_telemetry.py
Co-simulation builder	Write SUMO-CARLA helper commands without starting CARLA automatically	cosim/run_synchronization.sh, cosim/run_synchronization.bat
Analysis	Parse SUMO and optional CARLA outputs, then export tables and figures	scenario_summary.csv, mobility_energy_table.csv, and analysis/figures/

1 Network Preparation

2 The network preparation step uses standard-library XML parsing to summarize the OpenDRIVE file without
3 loading all simulator dependencies. The summary counts roads, junctions, lane sections, lanes, signals, objects, and
4 controllers. It also records sample road and junction identifiers and the OpenDRIVE header. This simple summary is
5 not a full semantic validation of OpenDRIVE, but it gives reviewers and analysts a transparent check that the expected
6 network was used.

7 SUMO Project Generation

8 The SUMO builder supports two input modes. If SUMO files already exist beside the OpenDRIVE file or
9 are named in the scenario configuration, the builder copies those files into the project. This is useful when a case
10 study already has a calibrated SUMO network and route file. If those files are not available, the builder plans or runs
11 `netconvert` to create a SUMO network from the prepared OpenDRIVE file and plans or runs `randomTrips.py` for
12 test demand generation. Missing SUMO tools are nonfatal in dry-run builds and fatal only when the user requests a
13 real simulation run.

14 The builder writes vehicle types for internal-combustion and electric passenger vehicles. The electric vehicle
15 type includes SUMO battery-device parameters, while the internal-combustion type uses a configurable emission
16 class. A scenario file controls electric-vehicle share, maximum speed, driver sigma, simulation start and end time,
17 and output-file options. When route records are generated by the workflow, vehicle types are assigned using a seeded
18 random number generator so the same scenario can be rebuilt reproducibly.

19 CARLA and Co-Simulation Project Generation

20 The CARLA builder writes Python scripts instead of directly controlling the CARLA server during project
21 generation. The loading script connects to the configured host and port, reads `network/prepared.xodr`, calls
22 CARLA's OpenDRIVE world-generation API, and applies fixed-time-step settings. The telemetry script records frame
23 number, elapsed time, actor identifiers, pose, velocity, acceleration, speed, acceleration magnitude, and jerk to a CSV
24 file. Optional ego-vehicle filters can restrict the telemetry file to selected actors.

25 The co-simulation builder creates shell and Windows batch scripts for CARLA's SUMO synchronization
26 helpers. The scripts create CARLA-compatible SUMO vehicle types, convert the OpenDRIVE network for synchro-
27 nization, and run the synchronization process with a configurable time step and traffic-light manager. The generated
28 scripts require `CARLA_HOME`; they intentionally do not start the CARLA server automatically.

29 Analysis

30 The analysis module parses SUMO `tripinfo.xml` with streaming XML parsing so that large output files can
31 be processed without loading the entire document tree. Nested `emissions` and `battery` records are flattened when
32 available. The analysis reports completed vehicles, average travel time, route length, average speed, waiting time, time

1 loss, throughput, total distance, fuel energy, electricity use, battery energy consumed, regenerated energy, and energy
 2 intensity. CARLA telemetry summaries are written only when a telemetry CSV is available; otherwise the analysis
 3 metadata records why CARLA results were skipped.

4 CASE STUDY

5 Network and Scenario

6 The demonstration uses the Tempe network bundled under `paper/examples/datasets/tempe_net`. The
 7 prepared OpenDRIVE file reports a rectangular header extent of 9,771.55 m east-west and 15,721.36 m north-south.
 8 The case study is intended as a workflow validation example rather than a calibrated policy evaluation. Table 2 gives
 9 the OpenDRIVE network counts written by the workflow.

TABLE 2 Tempe OpenDRIVE Network Summary

Network element	Count
Roads	5,885
Junctions	755
Lane sections	5,885
Lanes	15,496
Signals	4,149
Controllers	243
Objects	0

10 The scenario configuration files include a baseline case, electric-vehicle fleet share cases, an eco-speed case,
 11 and a signal-optimized placeholder case. The paper demonstration uses the existing SUMO files associated with the
 12 Tempe network and the SUMO-only smoke-run configuration. That configuration runs a 10-minute window from
 13 27,000 s to 27,600 s and disables high-volume emissions, battery, and floating-car output files to keep the repository
 14 example small.

15 Commands

16 The dry-run build command creates SUMO, CARLA, and co-simulation files without requiring a CARLA
 17 server:

```
18 python -m pyopendrive.sim.cli build \
19 --xodr paper/examples/datasets/tempe_net/tempe.xodr \
20 --out paper/examples/sumo_carla_energy/generated/tempe_baseline \
21 --scenario paper/examples/sumo_carla_energy/configs/scenario_baseline.yaml \
22 --build-sumo --build-carla --build-cosim --dry-run --overwrite
```

23 The SUMO-only command builds the project, executes SUMO when it is available, and analyzes the outputs:

```
24 python -m pyopendrive.sim.cli sumo-only \
25 --xodr paper/examples/datasets/tempe_net/tempe.xodr \
26 --out paper/examples/sumo_carla_energy/generated/tempe_sumo_only \
27 --scenario paper/examples/sumo_carla_energy/configs/scenario_sumo_from_xodr.yaml \
28 --overwrite
```

29 RESULTS

30 Generated Files

31 The workflow generated the expected project folders and reproducibility files. For the SUMO-only example,
 32 the project includes the copied and prepared OpenDRIVE network, a network summary JSON file, copied SUMO
 33 network and route inputs, a generated SUMO configuration, vehicle-type definitions, a run command log, and analysis
 34 tables and figures. CARLA and co-simulation scripts are generated by the dry-run build, but CARLA telemetry is not
 35 interpreted unless the CARLA runtime is actually executed.

36 SUMO-Only Mobility Output

37 Table 3 reports the smoke-run SUMO mobility summary. The run completed two selected vehicles in the
 38 10-minute example window. Because detailed energy, emissions, battery, and floating-car outputs were intentionally

- 1 disabled for this small repository run, the energy and emissions columns in the analysis files should be interpreted as
 2 unavailable for this demonstration, not as zero-energy vehicle operation.

TABLE 3 SUMO-Only Smoke-Run Mobility Metrics

Metric	Value
Completed vehicles	2
Average travel time, s	9.0
Average route length, m	148.2
Average speed, m/s	16.47
Average waiting time, s	0.0
Average waiting count	0.0
Average time loss, s	0.0
Throughput, vehicles/h	720.0
Total completed distance, km	0.296
Detailed energy and emissions	Not analyzed in smoke run
CARLA telemetry	Not executed in SUMO-only run

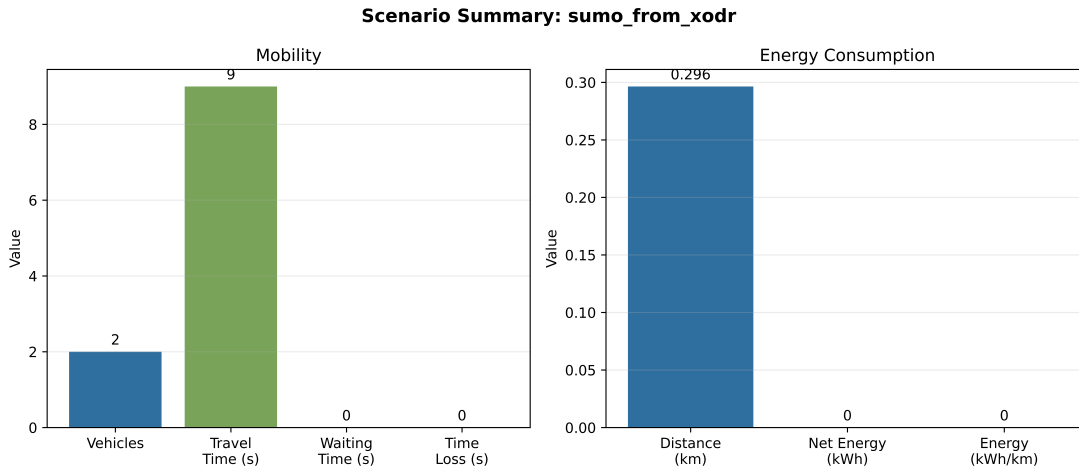


Figure 1 Automatically generated mobility-energy summary figure for the SUMO-only Tempe smoke run. Energy values are not interpreted for this run because detailed energy outputs were disabled in the scenario configuration.

- 3 Figure 1 shows the generated figure artifact. The figure is useful as a reproducibility check: the same
 4 analysis command writes the CSV tables and the plotted summary in a deterministic location under the project's
 5 analysis/figures directory. For a final policy-facing study, the same plotting path can be used after enabling
 6 detailed SUMO energy and emission outputs and running the scenario set across multiple random seeds.

7 CARLA and Co-Simulation Readiness

- 8 The CARLA and co-simulation parts of the workflow were evaluated through file generation and dependency
 9 checks. The generated CARLA files include a script for loading the prepared OpenDRIVE network into a CARLA
 10 world and a script for recording vehicle telemetry. The generated co-simulation scripts include commands for CARLA
 11 SUMO vehicle-type creation, CARLA-compatible SUMO network conversion, and synchronization execution. When
 12 the CARLA Python API or CARLA_HOME is absent, the command-line interface reports these missing dependencies
 13 with explicit messages and does not claim that CARLA results are available.

14 DISCUSSION

- 15 The demonstration shows that PyOpenDRIVE can serve as a practical workflow layer between an OpenDRIVE
 16 network and simulator-specific project files. This is useful for transportation studies because many reproducibility

failures happen outside the modeling equations: analysts may lose track of which network was used, which simulator command generated an output, or which external dependency was needed. The project layout and JSON command logs address those problems in a simple way that can be reviewed without specialized software.

The current implementation is intentionally conservative about claims. SUMO is the default source for mobility, emission, and energy measures because its output files include established trip, emissions, and battery records. CARLA is used for three-dimensional simulation and telemetry unless an additional validated energy model is attached. The analysis also avoids fabricating cross-simulator consistency metrics. If vehicle matching between SUMO and CARLA is unavailable, the workflow reports the CARLA fields as unavailable rather than estimating unmatched speed or position differences.

The main limitation of the demonstration is that the reported Tempe run is a smoke test, not a calibrated multi-scenario evaluation. It verifies that the network preparation, SUMO execution, parsing, and paper-output generation work on a real project directory. It does not support conclusions about the energy effects of electrification, eco-speed operation, or signal optimization. A final TRB study using this workflow should run the full scenario set, enable detailed energy and emissions outputs, calibrate demand and vehicle parameters, and use multiple random seeds.

Another limitation is simulator interpretation. OpenDRIVE improves network exchange, but it does not guarantee that SUMO and CARLA will represent all junctions, signals, lanes, and surface geometry identically. The workflow makes these translations easier to inspect, but final validation should compare network topology, route feasibility, travel times, and selected trajectory statistics after both simulators are run.

CONCLUSIONS

This paper presents an OpenDRIVE-based PyOpenDRIVE workflow for preparing SUMO, CARLA, and optional SUMO-CARLA co-simulation projects. The workflow copies and summarizes a source OpenDRIVE network, writes reproducible project metadata, generates simulator-specific files and run scripts, and exports analysis tables and figures. The Tempe example demonstrates that the workflow can prepare a network with 5,885 roads and 15,496 lanes, run a SUMO-only smoke test, and write reviewable mobility outputs and figure artifacts.

The workflow is most useful as a reproducible simulation-preparation and analysis layer for transportation digital-twin studies. Its current results support workflow feasibility, not policy conclusions. Future work should add batch scenario execution, calibrated demand inputs, multi-seed statistical summaries, CARLA trajectory validation, and direct comparison of SUMO and CARLA vehicle traces where stable vehicle matching is available.

ACKNOWLEDGMENTS

The author thanks the open-source SUMO, CARLA, ASAM OpenDRIVE, and libOpenDRIVE communities for the tools and standards that make this workflow possible.

AI disclosure: The author used OpenAI ChatGPT 5.5 to polish the manuscript text and LaTeX formatting from repository files and official author instructions. The author is responsible for verifying the technical claims, references, and final submission metadata before submission.

AUTHOR CONTRIBUTIONS

The author confirms contribution to the paper as follows: study conception and design: X. Luo; software implementation and workflow integration: X. Luo; data collection and example execution: X. Luo; analysis and interpretation of results: X. Luo; draft manuscript preparation: X. Luo. The author reviewed the results and approved the final version of the manuscript.

DECLARATION OF CONFLICTING INTERESTS

The author declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

FUNDING

The author disclosed no financial support for the research, authorship, and/or publication of this article.

REFERENCES

- 1 **REFERENCES**
- 2 Álvarez López, Pablo, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich,
- 3 Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. 2018. “Microscopic Traffic
- 4 Simulation Using SUMO.” In *2018 21st International Conference on Intelligent Transportation Systems*
- 5 *(ITSC)*, 2575–2582. IEEE. <https://doi.org/10.1109/ITSC.2018.8569938>.
- 6 ASAM e.V. 2026. *ASAM OpenDRIVE 1.9.0*. Accessed May 31, 2026. <https://www.asam.net/standards/detail/opendrive/>
- 7 *e/*.
- 8 Ashayer, Sima, Firas Elhag, Pengyuan Sun, and Mina Sartipi. 2025. “Co-Simulation in Intelligent Transportation
- 9 Systems: A Survey Targeting Connected and Autonomous Vehicles.” *Transportation Research Record* 2679
- 10 (10). <https://doi.org/10.1177/03611981251342248>.
- 11 Azfar, Talha, Kaicong Huang, Andrew Tracy, Sandra Misiewicz, Chenxi Liu, and Ruimin Ke. 2025. “Traffic Co-
- 12 Simulation Framework Empowered by Infrastructure Camera Sensing and Reinforcement Learning.” *Journal*
- 13 *of Intelligent Transportation Systems*. Published online September 15, 2025. [https://doi.org/10.1080/154724](https://doi.org/10.1080/15472450.2025.2559410)
- 14 [50.2025.2559410](https://doi.org/10.1080/15472450.2025.2559410).
- 15 Borchers, Philipp, Tjark Koopmann, Lukas Westhofen, Jan Steffen Becker, Lina Putze, Dominik Grundt, Thies
- 16 de Graaff, Vincent Kalwa, and Christian Neurohr. 2025. “TSC2CARLA: An Abstract Scenario-Based Ver-
- 17 ification Toolchain for Automated Driving Systems.” *Science of Computer Programming* 242: 103256.
- 18 <https://doi.org/10.1016/j.scico.2024.103256>.
- 19 Cao, Yue, Wei Shangguan, Arnoud Visser, Junjie Chen, Linguo Chai, and Baigen Cai. 2025. “Segment-Based Trajectory
- 20 Prediction and Risk Assessment for RSU-Assisted CAVs at Signalized Intersections.” *IEEE Transactions on*
- 21 *Intelligent Vehicles* 10 (1): 427–445. <https://doi.org/10.1109/TIV.2024.3414198>.
- 22 CARLA Simulator Project. 2026. *CARLA Documentation: OpenDRIVE Standalone Mode and SUMO Co-Simulation*.
- 23 Accessed May 31, 2026. <https://carla.readthedocs.io/en/latest/>.
- 24 Chen, Yanzhan, Liang Zheng, and Zhen Tan. 2024. “Roadside LiDAR Placement for Cooperative Traffic Detection by
- 25 a Novel Chance Constrained Stochastic Simulation Optimization Approach.” *Transportation Research Part*
- 26 *C: Emerging Technologies* 167: 104838. <https://doi.org/10.1016/j.trc.2024.104838>.
- 27 Dosovitskiy, Alexey, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. 2017. “CARLA: An Open
- 28 Urban Driving Simulator.” In *Proceedings of the 1st Annual Conference on Robot Learning*, 1–16. PMLR.
- 29 <https://proceedings.mlr.press/v78/dosovitskiy17a.html>.
- 30 Eclipse SUMO Project. 2026. *SUMO User Documentation*. Accessed May 31, 2026. <https://sumo.dlr.de/docs/>.
- 31 Han, Xu, Runsheng Xu, Xin Xia, Anoop Sathyan, Yi Guo, Pavle Bujanovic, Ed Leslie, Mohammad Goli, and Jiaqi
- 32 Ma. 2022. “Strategic and Tactical Decision-Making for Cooperative Vehicle Platooning with Organized
- 33 Behavior on Multi-Lane Highways.” *Transportation Research Part C: Emerging Technologies* 145: 103952.
- 34 <https://doi.org/10.1016/j.trc.2022.103952>.
- 35 Li, Yueyuan, Wei Yuan, Songan Zhang, Weihao Yan, Qiyuan Shen, Chunxiang Wang, and Ming Yang. 2024. “Choose
- 36 Your Simulator Wisely: A Review on Open-Source Simulators for Autonomous Driving.” *IEEE Transactions*
- 37 *on Intelligent Vehicles* 9 (5): 4861–4876. <https://doi.org/10.1109/TIV.2024.3374044>.
- 38 libOpenDRIVE Contributors. 2023. *libOpenDRIVE: Small, Lightweight C++ Library for Handling OpenDRIVE Files*.
- 39 Accessed May 31, 2026. <https://github.com/pageldev/libOpenDRIVE>.
- 40 Silva, Ivo, Helder Silva, Fabricio Botelho, and Cristiano Pendao. 2024. “Realistic 3D Simulators for Automotive: A
- 41 Review of Main Applications and Features.” *Sensors* 24 (18): 5880. <https://doi.org/10.3390/s24185880>.
- 42 Varga, Balazs, Daniel Doba, and Tamas Tettamanti. 2023. “Optimizing Vehicle Dynamics Co-Simulation Performance
- 43 by Introducing Mesoscopic Traffic Simulation.” *Simulation Modelling Practice and Theory* 125: 102739.
- 44 <https://doi.org/10.1016/j.simpat.2023.102739>.
- 45 Xu, Runsheng, Yi Guo, Xu Han, Xin Xia, Hao Xiang, and Jiaqi Ma. 2021. “OpenCDA: An Open Cooperative
- 46 Driving Automation Framework Integrated with Co-Simulation.” In *2021 IEEE International Intelligent*
- 47 *Transportation Systems Conference (ITSC)*, 1155–1162. IEEE. [https://doi.org/10.1109/ITSC48978.2021.956](https://doi.org/10.1109/ITSC48978.2021.9564825)
- 48 [4825](https://doi.org/10.1109/ITSC48978.2021.9564825).
- 49 Xu, Runsheng, Hao Xiang, Xu Han, Xin Xia, Zonglin Meng, Chia-Ju Chen, Camila Correa-Jullian, and Jiaqi Ma. 2023.
- 50 “The OpenCDA Open-Source Ecosystem for Cooperative Driving Automation Research.” *IEEE Transactions*
- 51 *on Intelligent Vehicles* 8 (4): 2698–2711. <https://doi.org/10.1109/TIV.2023.3244948>.
- 52 Zheng, Zhaoliang, Xu Han, Xin Xia, Letian Gao, Hao Xiang, and Jiaqi Ma. 2023. “OpenCDA-ROS: Enabling Seamless
- 53 Integration of Simulation and Real-World Cooperative Driving Automation.” *IEEE Transactions on Intelligent*
- 54 *Vehicles* 8 (7): 3775–3780. <https://doi.org/10.1109/TIV.2023.3298528>.